Reinforcement Learning

First Assignment

Federica Lavini

Matricola: 1941986

Contents

1 Theory **3**
1.1 Convergence of Policy Iteration 3
1.2 Value Iteration 4

2 Practice **6**
2.1 FrozenLake Gymnasium: Policy Iteration 6
2.2 Pendulum: iLQR 7

3 Collaborations **8**

1 Theory

1.1 Convergence of Policy Iteration

Consider a finite MDP defined by the tuple $(\mathcal{S}, \mathcal{A}, p, R, \gamma)$, with state set $\mathcal{S}$, action set $\mathcal{A}$, transition probabilities $p(s' \mid s, a)$, reward function $r(s, a, s')$, discount factor $0 \leq \gamma < 1$ and a deterministic stationary policy $\pi : \mathcal{S} \to \mathcal{A}$. The value of a policy π is the unique solution of

$$V^{\pi}(s) = \sum_{s' \in \mathcal{S}} p(s' \mid s, \pi(s)) \Big(r(s, \pi(s), s') + \gamma V^{\pi}(s') \Big), \quad \forall s \in \mathcal{S}. \tag{1}$$

Define the Bellman operator of π as

$$(T^{\pi} V)(s) = \sum_{s'} p(s' \mid s, \pi(s)) \Big(r(s, \pi(s), s') + \gamma V(s') \Big), \tag{2}$$

and the optimal Bellman operator

$$(TV)(s) = \max_{a \in \mathcal{A}} \sum_{s'} p(s' \mid s, a) \Big(r(s, a, s') + \gamma V(s') \Big). \tag{3}$$

Policy Iteration

A sequence of π_k is constructed by a Policy Iteration, alternating:

1. **Policy Evaluation:** compute V^{π_k} such that $V^{\pi_k} = T^{\pi_k} V^{\pi_k}$;
2. **Policy Improvement:** define π_{k+1} by

$$\pi_{k+1}(s) \in \arg\max_{a \in \mathcal{A}} \sum_{s'} p(s' \mid s, a) \Big(r(s, a, s') + \gamma V^{\pi_k}(s') \Big).$$

Lemma

If π' selects the best actions based on V^{π}, that means

$$(T^{\pi'} V^{\pi})(s) \geq V^{\pi}(s), \quad \forall s, \tag{4}$$

then π' performs at least as well as π in all states, and strictly better in at least one state if $\pi' \neq \pi$.

Proof. Since π' is chosen to improve V_{π}, applying it every time makes the value function always increase (or stay the same) until it reaches $V_{\pi'}$. So, $V_{\pi'}$ is at least as large as V_{π} in every state. If $\pi' \neq \pi$, then it is better in at least one state.

Monotonic Improvement

Applying the lemma to $\pi = \pi_k$ and $\pi' = \pi_{k+1}$ we have

$$V^{\pi_{k+1}} \geq V^{\pi_k}, \tag{5}$$

and if $\pi_{k+1} \neq \pi_k$, then the value improves in at least one state. So, the sequence $\{V^{\pi_k}\}$ does not decrease and may increase in every state.

Finite Convergence

Since the MDP has a finite number of states and actions, there are only many deterministic policies that are finite. Policy Iteration creates a sequence of policies $\{\pi_k\}$, and every time the policy changes, the value function gets better. So, the algorithm must eventually stop after a finite number of steps at a policy π^* that is greedy with respect to its own value function:

$$TV_{\pi^*} = V_{\pi^*}.$$

Since the Bellman optimality operator T has a unique fixed point V^*, we get $V_{\pi^*} = V^*$, which means π^* is optimal. So we proved that Policy Iteration converges in a finite number of steps to an optimal policy.

1.2 Value Iteration

I used the Value Iteration algorithm to compute $V_{k+1}(s_4)$ required in the problem.

Value Iteration Algorithm

Algorithm 1 Value Iteration

```
procedure ValueIteration(S, A, P, R, γ, ε)
    Initialize V(s) = 0 for all s ∈ S
    repeat
        Δ ← 0
        for each state s ∈ S do
            v ← V(s)
            V(s) ← max_{a∈A} [Σ_{s'} p(s'|s,a) (R(s,a,s') + γV(s'))]
            Δ ← max(Δ, |v − V(s)|)
        end for
    until Δ < ε
    return V
end procedure
```

where the Value Iteration update rule for each state s is:

$$V_{k+1}(s) = \max_{a \in \mathcal{A}} \left[\sum_{s'} p(s'|s,a) \left(r(s,a,s') + \gamma V_k(s') \right) \right] \tag{6}$$

where:

- $\mathcal{S}$ is the set of states
- $\mathcal{A}$ is the set of actions
- $p(s'|s,a)$ is the transition probability
- $r(s,a,s')$ is the reward function
- γ is the discount factor ($0 \leq \gamma < 1$)
- ϵ is the convergence threshold

Considering now the giving problem:

- States: $\{s_i : i \in \{1, \ldots, 5\}\}$
- Reward:
$$r(s,a,s') = \begin{cases} 1 & \text{if } s = s_3 \\ -10 & \text{if } s = s_5 \\ 0 & \text{otherwise} \end{cases}$$
- Dynamics: $p(s_4|s_4,a_1) = 0.8$, $p(s_5|s_4,a_1) = 0.2$
- Policy: $\pi(s) = a_1 \quad \forall s \in S$

and the value function at the iteration $k = 1$:

$$v_k = [0, 0, 1, 0, -10]$$

with $\gamma = 0.8$.

From the problem, we know that the dynamics converge to the following immediate rewards:

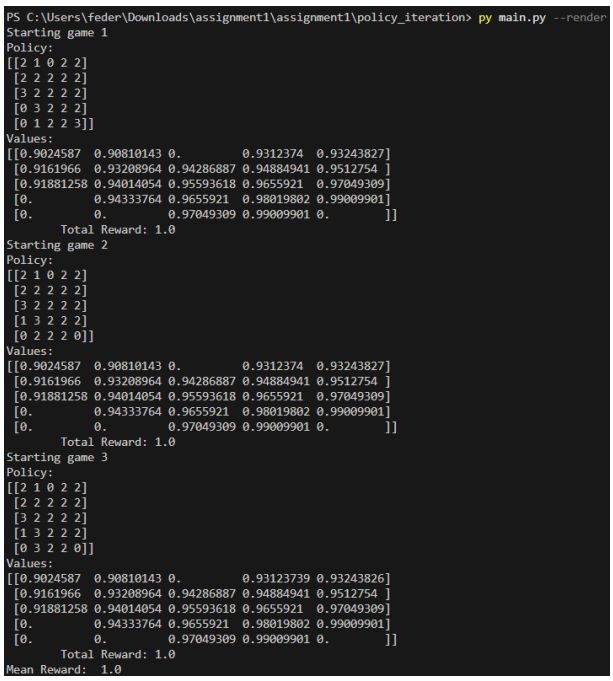
$$r(s_4, a_1, s_4) = 0, \quad r(s_4, a_1, s_5) = -10.$$

We now compute $V_{k+1}(s_4)$ with $k = 1$:

$$\begin{aligned}
V_2(s_4) &= \sum_{s'} p(s' \mid s_4, a_1)\,[r(s_4, a_1, s') + \gamma V_1(s')] \\
&= 0.8 \times [0 + 0.8 \times V_1(s_4)] + 0.2 \times [(-10) + 0.8 \times V_1(s_5)] \\
&= 0.8 \times [0 + 0.8 \times 0] + 0.2 \times [(-10) + 0.8 \times (-10)] \\
&= 0.8 \times 0 + 0.2 \times (-18) \\
&= 0 - 3.6 \\
&= -3.6
\end{aligned}$$

2 Practice

2.1 FrozenLake Gymnasium: Policy Iteration



```
PS C:\Users\feder\Downloads\assignment1\assignment1\policy_iteration> py main.py --render
Starting game 1
Policy:
[[2 1 0 2 2]
 [2 2 2 2 2]
 [3 2 2 2 2]
 [0 3 2 2 2]
 [0 1 2 2 3]]
Values:
[[0.9024587  0.90810143 0.         0.9312374  0.93243827]
 [0.9161966  0.93208964 0.94286887 0.94884941 0.9512754 ]
 [0.91881258 0.94014054 0.95593618 0.9655921  0.97049309]
 [0.         0.94333764 0.9655921  0.98019802 0.99009901]
 [0.         0.         0.97049309 0.99009901 0.        ]]
        Total Reward: 1.0
Starting game 2
Policy:
[[2 1 0 2 2]
 [2 2 2 2 2]
 [3 2 2 2 2]
 [1 3 2 2 2]
 [0 2 2 2 0]]
Values:
[[0.9024587  0.90810143 0.         0.9312374  0.93243827]
 [0.9161966  0.93208964 0.94286887 0.94884941 0.9512754 ]
 [0.91881258 0.94014054 0.95593618 0.9655921  0.97049309]
 [0.         0.94333764 0.9655921  0.98019802 0.99009901]
 [0.         0.         0.97049309 0.99009901 0.        ]]
        Total Reward: 1.0
Starting game 3
Policy:
[[2 1 0 2 2]
 [2 2 2 2 2]
 [3 2 2 2 2]
 [1 3 2 2 2]
 [0 3 2 2 0]]
Values:
[[0.9024587  0.90810143 0.         0.93123739 0.93243826]
 [0.9161966  0.93208964 0.94286887 0.94884941 0.9512754 ]
 [0.91881258 0.94014054 0.95593618 0.9655921  0.97049309]
 [0.         0.94333764 0.9655921  0.98019802 0.99009901]
 [0.         0.         0.97049309 0.99009901 0.        ]]
        Total Reward: 1.0
Mean Reward:  1.0
```

Figure 1: Caption describing the image.

The goal of the first exercise is to implement the Policy Iteration algorithm. The algorithm guides the agent to the terminal state, avoiding obstacles and maximizing the reward. In particular, the method alternates between two main steps:

1. **Policy Evaluation**: computing $V^{\pi}(s)$ for the current policy
2. **Policy Improvement**: updating the policy, choosing the action that maximizes the expected value

The two steps are repeated until the policy does not change anymore.

The algorithm starts from a random initial policy:

```
policy = np.random.randint(0, env.action_space.n, env.observation_space.n)
values = np.zeros(env.observation_space.n)
```

The random policy is used to initialize the random action for each state and avoid initial knowledge about the environment. A `STATES` matrix is also created to map the indices used by library to the grid coordinates.

During the Policy Evaluation phase, the Bellman is applied:

$$V(s) = \sum_{s'} P(s'|s, \pi(s)) \left[r(s, s') + \gamma V(s') \right]$$

Values of each phase are iteratively updated until the maximum difference between two iterations Δ is under a threshold θ:

```
next_prob = transition_probabilities(env, state, policy[i], ...)
values[i] = (next_prob * (REWARDS + gamma * old_val)).sum()
```

Terminal states and frozen holes are set to zero, they have no future transitions.

After value convergence, in the Policy Improvement step only the highest expected value is selected. The selection is made after the evaluation of all actions. If no action changes the policy, it is marked as `stable` and the algorithm has converged to π^*.

Policy Iteration converges quickly in the FrozenLake environment because the state space is small, the transition model is known, and an exact policy evaluation is performed at each iteration. The agent can

navigate the grid, avoid obstacles, and reach the goal. At the same time the computation is heavy due to repeated policy evaluations From the terminal, image 1, we observe that the agent reaches the goal every time, achieving a mean reward of 1.0. The value function increases near the goal. Small differences in final policies may occur due to stochastic transitions and random map initialization.

2.2 Pendulum: iLQR

The goal of this exercise is to implement the iLQR (Iterative Linear Quadratic Regulator) algorithm to control the pendulum in the Pendulum-v1 environment from Gymnasium. We want the pendulum to be in a vertical position, with $\theta = 0$ rad and a small angular velocity.

Usually the iLQR algorithm is an iterative optimal control for non-linear systems. It works by linearizing the dynamics of the current trajectory, approximating the cost with a local quadratic, and solving a linear-quadratic problem to update the controls. Repeating this process iteratively, the trajectory converges to an optimal solution.

The discrete dynamics of the pendulum is:

```
newthdot = thdot + (3.0*g/(2.0*l) * np.sin(th) + 3.0/(m*l**2)*u) * dt
newth = th + newthdot * dt
newthdot = np.clip(newthdot, -8, 8)
x = np.array([newth, newthdot])
```

The angular velocity is in the range $[-8, 8]$ for stabilization.

In the backward phase for each step we calculate:

```
inversa = np.linalg.inv(Rt + Bt.T @ Pt1 @ Bt)
kt = -inversa @ (rt + Bt.T @ pt1)
Kt = -inversa @ (Bt.T @ Pt1 @ At)

pt = qt + Kt.T @ (Rt @ kt + rt) \
     + (At + Bt @ Kt).T @ pt1 \
     + (At + Bt @ Kt).T @ Pt1 @ Bt @ kt

Pt = Qt + Kt.T @ Rt @ Kt \
     + (At + Bt @ Kt).T @ Pt1 @ (At + Bt @ Kt)
```

that are the feedback gain K_t and feedforward k_t.

Then the trajectory is updated using the following law in the forward step:

```
control = k_seq[t] + K_seq[t] @ (x_seq_hat[t] - x_seq[t])
u_seq_hat[t] = np.clip(u_seq[t] + control, -2, 2)
x_seq_hat[t+1] = self.f(x_seq_hat[t], u_seq_hat[t])
```

Then the dynamics is applied to get x_{t+1}.

The cost function used is:

```
costs = angle_normalize(x[0])**2 + 0.1*x[1]**2 + 0.001*(u**2)
```

Here, θ is normalized between $[-\pi, \pi]$.

Example of output of the program:

```
Episode 0 reward: -124.45
Episode 1 reward: -245.27
Episode 2 reward: -374.75
Average reward: -248.16
```

The rewards are negative because in Gymnasium the reward is the negative cost. The iLQR controller is able to move the pendulum closer to the vertical position with good results.

Results change slightly between runs because the initial state of the pendulum is random.

3 Collaborations

I discussed the assignment with Flaminia Lanzellotti, matricola number 2201485.